zk-UAV-Proximity-Proof

ゼロ知識証明によるプライバシー保護型ドローン近接検知システム

到達フェーズ	開発体制	開発期間	担当
PoC（実機デモ完成）	個人開発	約3ヶ月（2026年2月〜5月）	企画・設計・実装

GitHub: https://github.com/SMRI2170/zerosky-opendroneid Dart 52.9% / Python 32.3% / Circom 8.3% / Shell 5.8%

背景と課題

2022年施行の改正航空法によりドローンの機体登録・リモートIDが義務化され、機体ID・位置情報のリアルタイム配信（BLE/Wi-Fi）が始まった。空の安全管理は一歩前進したが、同時に新たな問題が浮上した。

特定のドローンが「自分の周囲30m以内に5秒以上滞空していた」という事実を、自分の位置情報を一切開示せずに第三者へ証明する手段がないという点だ。

証明者の実座標を秘匿したまま近接の事実だけを数学的に証明できれば、セキュリティ・損害保険・民事訴訟など、各場面での信頼できる根拠として機能する。これがプロジェクトの出発点となった。

解決アプローチ

アプローチ	採用技術	要点
完全エッジ完結型 ZKP	Circom + Groth16 rapidsnark (Rust) circom-witnesscalc	クラウドを一切使わず、Android端末上でWitness計算・Groth16証明生成・検証まで完結させた。30m以内かつ5秒以内という条件を専用回路で判定し、実座標は証明に含めない。
【通信】 ドローン位置の取得	OpenDroneID (ASTM F3411-22a)	公開BLE放送から機体ID・位置を受信。opendroneid-core-c参照実装でバイトレイアウトを照合し、規格準拠を確認済み。
【空間】 3D位置の取得	Android GPS + 気圧センサー	GPSに加え気圧センサーで相対高度を算出し、平面だけでなく高度差も考慮した3D近接判定を実現。
【視覚】 エッジAI検知	YOLOv8-tiny TFLite (VisDrone転移学習 →drone fine-tuning)	VisDrone（空撮小物体検知）で転移学習後、drone単一クラスにfine-tuning。証明前にリアルタイムでドローンを検知し視覚的証拠として記録する。

主な成果

項目	内容
ZKP回路の最適化	距離と時間を同時判定するCircom回路を自作。--O2最適化でconstraint数を削減し、モバイル端末での実行速度を改善（最適化比較レポート付き）。
Androidでの証明生成	rapidsnark（Rustネイティブ）をFlutterに直接組み込み、Android実機上でGroth16証明パイプライン全体を動作させることに成功した。
実機PoC完了	Raspberry Pi（GPS+BME280）からBLE送信、Android端末で受信・AI検知・ZKP生成まで繋がるフルパイプラインを屋外環境で実証した。
OpenDroneID準拠	ASTM F3411-22a のByte1 nibble順・EWDirection・SpeedMult等をopendroneid-core-c照合で修正し、実機との通信を確認済み。

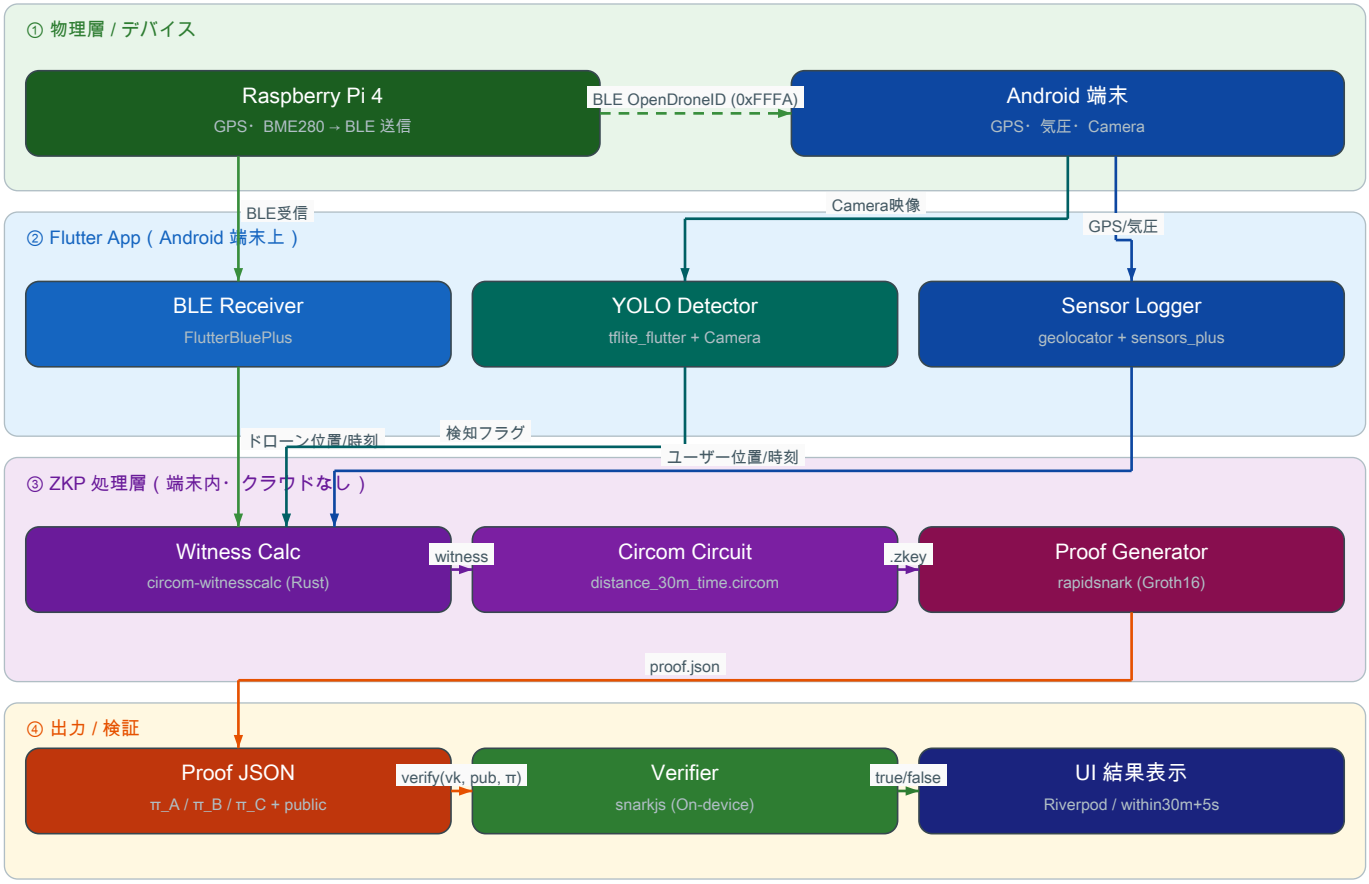
今後の展望

本番化	MPC（Multi-Party Computation）セレモニーによる Trusted Setup の実施を計画している。
検知強化	独自データの継続収集と再 Fine-tuning により、より多様な機体・環境への対応を進める。
iOS 展開	rapidsnark の iOS ビルドを整備し、Android・iOS 両プラットフォームへの対応を目指す。

システムアーキテクチャ概要

処理フロー・コンポーネント構成・セキュリティ / 運用の工夫

システム構成図（完全エッジ完結・クラウドなし）



コンポーネント一覧

レイヤー	コンポーネント	技術	役割
物理	Raspberry Pi 4	Python / BlueZ	GPS・BME280取得 → BLE OpenDroneID配信
物理	Android 実機	Flutter / Android SDK	BLE受信・カメラ・GPS・気圧センサーのホスト
Edge	BLE Receiver	flutter_blue_plus	OpenDroneID (0xFFFA) / Mfr AD (0x02E5) の解析
Edge	YOLO Detector	tflite_flutter IsolateInterpreter	YUV420→RGB変換・YOLOv8-tiny推論・Riverpod状態管理
Edge	Sensor Logger	geolocator sensors_plus	GPS 3D位置 + 気圧センサーによる相対高度取得
ZKP	Witness Calc	circom-witnesscalc (Rust FFI)	回路入力からwitnessファイルを生成
ZKP	Circom Circuit	Circom 2.x	30m以内 & 5秒以内 判定回路（--O2最適化）
ZKP	Proof Generator	rapidsnark (Rust Native)	Groth16証明生成をモバイル端末上で実行
出力	Verifier	snarkjs (Dart FFI)	vk.json と public signals を照合して真偽を返す
出力	UI (Riverpod)	flutter_riverpod 3.x	NotifierProvider で状態管理、証明結果を表示

セキュリティ・運用面の工夫

観点	内容

プライバシー	ZKPにより実座標を一切開示せず近接事実のみを証明。vk.jsonを公開すれば誰でも検証できる。
証明の健全性	Groth16の soundness により、不正な witnessから有効な証明は生成不能。回路内でassertにより入力範囲を強制。
BLE信頼性	Service UUID 0xFFFA + ASTM F3411-22aのバイト仕様に準拠。opendroneid-core-cで照合済み。
完全オフライン	クラウド不要で端末完結。外部通信が発生しないため通信傍受・サーバー障害リスクがない。
リソース管理	Riverpod autoDispose + IsolateInterpreterで画面離脱時にTFLite・BLEを即解放。メモリリーク対策済み。
Pi安定起動	hciconfig hci0 upはアダプタ起動済みでも exit code 1 を返すため、check=Trueを外して冪等化。

機械学習モデルとデータセット

ドローン検知モデルは2段階の転移学習で構築している。直接ドローンを検知するデータで学習する前に、まず「小さな物体を検知する能力」自体を習得させることが精度向上の鍵となった。

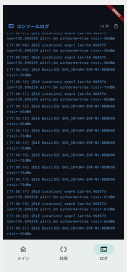
ステップ	データセット	役割と詳細	設定
① 転移学習（小物体検知）	VisDrone2019-DET（天津大学公開）	ドローン搭載カメラが上空から地上を撮影した映像に歩行者・自転車・車両など10クラスをアノテーションした空撮ベンチマーク。画面上で非常に小さく映る物体を正確に検知する能力をモデルに習得させることが目的で、この「小物体検知のベースライン」をYOLOv8 Nanoに転移させるために採用した。	base: yolov8n.pt size: 640px epochs: 80 batch: 8
② Fine-tuning（ドローン検知）	カスタムUAVデータ + Roboflow Drone Detection（代替・現在無効）	①で習得した「小物体を見つける目」を、ドローン1クラスのデータで上書き学習。ここで初めてドローンそのものを検知する能力を付与する。Roboflow Universe の Drone Detection データセットも代替候補として設定に組み込んでいる。	weights: ①best.pt epochs: 60 batch: 8 classes: 1（drone）
③ TFLite変換 + モバイル統合	—	ultralytics exportでfloat32 TFLiteに変換。tflite_flutter + IsolateInterpreterで非同期推論、GPU デリゲート（Android: GpuDelegateV2）で高速化。YUV420→RGB変換・NMS・スコア閾値0.20はFlutter側で処理。	format: tflite dtype: float32 delegate: GPU NMS IoU: 0.45

UIキャプチャ

実機スクリーンショット・実験風景



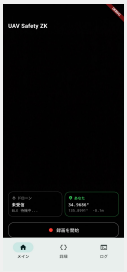
TFLite YOLO モデルによるドローン検知
（バウンディングボックスオーバーレイ表示）



OpenDroneID BLE パケット受信ログ（位置情報・機体ID をリアルタイム表示）



Google ML Kit フォールバック検知（YOLO 無効時の代替検知モード）



BLE 受信前 — 収集タブ（手入力モードの入力フォーム）



BLE 受信後 — 収集タブ（Pi から受信した位置情報を自動入力）



実験用ドローン（DJI Mini 系）（実機 PoC テスト環境）